

Teoría de la Computación 2026-1

May 5, 2026

Expediente	Nombre
223209096	Ahumada Herrera Jorge Alán
219208106	Bórquez Guerrero Angel Fernando
222206586	Juan Valenzuela Gael
223201053	Solis Zatarain Owen Adiel

Ejercicio 92. Diseñe dos AP diferentes que acepten por pila vacía al lenguaje aceptado por el AFD de «los tres focos» descrito en la figura 2.1.

	<i>a</i>	<i>b</i>	<i>c</i>
→   	  	  	  
  	  	  	  
  	  	  	  
  	  	  	  
  	  	  	  
  	  	  	  
*   	  	  	  
  	  	  	  

Figure 2.1: Tabla de transición del autómata de los tres focos.

Observación previa

El autómata de los tres focos acepta las cadenas en $\{a, b, c\}^*$ tales que cada foco termina encendido. Esto ocurre si y solo si cada símbolo aparece un número impar de veces.

$$L = \{w \in \{a, b, c\}^* : |w|_a, |w|_b, |w|_c \text{ son impares}\}$$

Este es un lenguaje regular, por lo que puede ser reconocido por un AP que simule al AFD.

Primer AP

Definimos el AP:

$$M_1 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$$

donde:

- $Q = \{q_{000}, q_{100}, q_{010}, q_{001}, q_{110}, q_{101}, q_{011}, q_{111}\}$
- $\Sigma = \{a, b, c\}$
- $\Gamma = \{Z_0\}$
- Estado inicial: q_{000}

Cada estado representa la paridad de (a, b, c) .

La pila nunca cambia (solo contiene Z_0).

Las transiciones son:

$$\delta(q, x, Z_0) = (q', Z_0)$$

donde q' cambia la paridad correspondiente.

Por ejemplo:

$$\delta(q_{000}, a, Z_0) = (q_{100}, Z_0)$$

$$\delta(q_{100}, a, Z_0) = (q_{000}, Z_0)$$

y análogamente para b y c .

Finalmente, se vacía la pila si se llega a q_{111} :

$$\delta(q_{111}, \varepsilon, Z_0) = (q_{111}, \varepsilon)$$

Por tanto, acepta por pila vacía.

Segundo AP

Construimos un AP distinto usando conteo en pila.

$$M_2 = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$$

donde:

- $Q = \{q\}$
- $\Sigma = \{a, b, c\}$
- $\Gamma = \{A, B, C, Z_0\}$

La idea es:

- Cada vez que aparece a , alternamos entre empujar y sacar A .
- Igual para b y c .

Ejemplo para a :
 $\delta(q, a, Z_0) = (q, AZ_0)$
 $\delta(q, a, A) = (q, \varepsilon)$

Análogamente para B y C .
Al final, la pila queda vacía si cada símbolo apareció un número par de veces de empuje/salida, lo que corresponde a impar en total (por construcción alternada).
Finalmente:
 $\delta(q, \varepsilon, Z_0) = (q, \varepsilon)$
Este AP es distinto porque usa la pila activamente.

Ejercicio 106. Pruebe que $A \xRightarrow{*} \gamma$ si y solo si cualesquiera $\alpha, \beta \in (T \cup V)^*$, $\alpha A \beta \xRightarrow{*} \alpha \gamma \beta$. Encuentre un contraejemplo para demostrar que en general no es cierto que si $\alpha A \beta \xRightarrow{*} \alpha \gamma \beta$ para algunas cadenas α y β entonces $A \xRightarrow{*} \gamma$.

Demostración

($\Rightarrow$) Suponga $A \equiv \gamma$

Por definición de equivalencia en gramáticas, A puede ser reemplazado por γ en cualquier contexto

Por tanto, para cualesquiera α, β :

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

($\Leftarrow$) Suponga que:

$$\forall \alpha, \beta, \alpha A \beta \Rightarrow \alpha \gamma \beta$$

Tomando $\alpha = \varepsilon$ y $\beta = \varepsilon$:

$$A \Rightarrow \gamma$$

Además, como la sustitución es válida en cualquier contexto, implica equivalencia estructural, por lo que:

$$A \equiv \gamma$$

Contraejemplo

Sea la gramática:

$$A \rightarrow a|b$$

$$\text{y } \gamma = a$$

Entonces:

$$A \Rightarrow a$$

pero también:

$$A \Rightarrow b$$

Ahora, para algunas cadenas (por ejemplo con $\alpha = \varepsilon, \beta = \varepsilon$):

$$A = a$$

pero no es cierto que:

$$A \equiv a$$

porque A genera más cadenas.

Esto demuestra que no basta que la propiedad se cumpla para algunas α, β .

Conclusión

Se construyeron dos AP distintos que aceptan el mismo lenguaje (uno simulando el AFD y el otro usando la pila activamente), y se probó formalmente la equivalencia del ejercicio 106 junto con un contraejemplo que muestra la necesidad de la condición universal.

Ejercicio 109. Una *gramática regular* (por la derecha) es una GLC que admite solo producciones de la forma:

$$A \rightarrow aB$$

$$C \rightarrow b$$

$$D \rightarrow \varepsilon$$

donde A, B, C, D son variables sintácticas y a, b son símbolos terminales. Encuentre una gramática regular para los siguientes conjuntos:

1. El conjunto de cadenas binarias que terminan en 111.

Gramática regular

Cada variable rastrea cuántos 1s consecutivos lleva la cadena al final:

S : cero, A : uno, B : dos, C : tres o más.

Definimos $G = (V, T, P, S)$ con $V = \{S, A, B, C\}$, $T = \{0, 1\}$.

Producciones P :

$$S \rightarrow 0S \mid 1A$$

$$A \rightarrow 0S \mid 1B$$

$$B \rightarrow 0S \mid 1C$$

$$C \rightarrow 0S \mid 1C \mid \varepsilon$$

Verificación:

$$S \Rightarrow 1A \Rightarrow 11B \Rightarrow 111C \Rightarrow 111 \quad (\text{acepta "111"})$$

$$S \Rightarrow 0S \Rightarrow 01A \Rightarrow 011B \Rightarrow 0111C \Rightarrow 0111 \quad (\text{acepta "0111"})$$

$$S \Rightarrow 1A \Rightarrow 11B \Rightarrow 110S \quad (S \text{ no deriva } \varepsilon, \text{ rechaza "110"})$$

2. $L_a = \{a^{2m+1} : m \geq 0\}$.

Gramática regular

Este lenguaje es el conjunto de cadenas con cantidad impar de a : $\{a, aaa, aaaaa, \dots\}$.

Dos variables que alternan la paridad.

Definimos $G = (V, T, P, S)$ con $V = \{S, A\}$, $T = \{a\}$.

Producciones P :

$$S \rightarrow aA$$

$$A \rightarrow aS$$

$$\mid \varepsilon$$

S = cantidad par de a leídas (inicio, 0 es par).

A = cantidad impar de a leídas (puede derivar ε).

Verificación:

$$S \Rightarrow aA \Rightarrow a \quad (\text{acepta } a, m = 0)$$

$$S \Rightarrow aA \Rightarrow aaS \Rightarrow aaaA \Rightarrow aaa \quad (\text{acepta } aaa, m = 1)$$

$$S \Rightarrow aA \Rightarrow aaS \quad (S \text{ no deriva } \varepsilon, \text{ rechaza } aa)$$

3. El conjunto de cadenas binarias con una cantidad impar de ceros y una cantidad impar de unos.

Gramática regular

Rastreamos la paridad de ambos símbolos simultáneamente ($2 \times 2 = 4$ combinaciones).

Definimos $G = (V, T, P, S)$ con $V = \{S, A, B, C\}$, $T = \{0, 1\}$.

- S : par de 0s, par de 1s (inicio).
- A : impar de 0s, par de 1s.
- B : par de 0s, impar de 1s.
- C : impar de 0s, impar de 1s (**aceptor**, deriva ε).

Producciones P :

$$\begin{array}{lcl} S & \rightarrow & 0A \\ & | & 1B \\ A & \rightarrow & 0S \\ & | & 1C \\ B & \rightarrow & 0C \\ & | & 1S \\ C & \rightarrow & 0B \\ & | & 1A \\ & | & \varepsilon \end{array}$$

Cada 0 cambia la paridad de los ceros; cada 1 cambia la paridad de los unos.

Solo C puede derivar ε (ambas paridades impares).

Verificación:

$S \Rightarrow 0A \Rightarrow 01C \Rightarrow 01$ (acepta "01": 1 cero, 1 uno)

$S \Rightarrow 1B \Rightarrow 10C \Rightarrow 10$ (acepta "10": 1 cero, 1 uno)

$S \Rightarrow 0A \Rightarrow 00S \Rightarrow 001B \Rightarrow 0011S$ (S no deriva ε , rechaza "0011")

4. L_{son} el lenguaje del ejemplo 15.

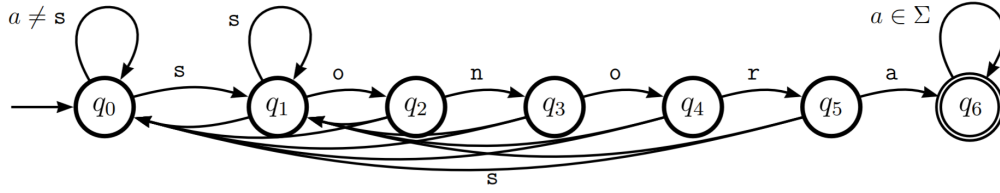


Figure 2.5: Autómata que lee sonora.

Gramática regular

Cada estado del AFD se convierte en una variable.

Definimos $G = (V, \Sigma, P, S)$ con $V = \{S, A, B, C, D, E, F\}$, $\Sigma = \{s, o, n, r, a\}$.

- $S (q_0)$: sin progreso (inicio).
- $A (q_1)$: leído **s**.
- $B (q_2)$: leído **so**.
- $C (q_3)$: leído **son**.
- $D (q_4)$: leído **sono**.
- $E (q_5)$: leído **sonor**.
- $F (q_6)$: leído **sonora** (aceptor, deriva ε).

Producciones P :

$S \rightarrow sA$	oS	nS	rS	aS
$A \rightarrow oB$	sA	nS	rS	aS
$B \rightarrow nC$	sA	oS	rS	aS
$C \rightarrow oD$	sA	nS	rS	aS
$D \rightarrow rE$	sA	oS	nS	aS
$E \rightarrow aF$	sA	oS	nS	rS
$F \rightarrow sF$	oF	nF	rF	$aF \mid \varepsilon$

La primera producción de cada variable avanza en la lectura de “sonora”.

Leer s siempre lleva a A (posible inicio de nuevo “sonora”).

Cualquier otra letra no esperada regresa a S .

F se queda en F con todo símbolo y puede derivar ε .

Verificación:

$S \Rightarrow sA \Rightarrow soB \Rightarrow sonC \Rightarrow sonoD \Rightarrow sonorE \Rightarrow sonoraF \Rightarrow sonora$

(acepta “sonora”)

$S \Rightarrow sA \Rightarrow ssA \Rightarrow ssoB \Rightarrow ssonC \Rightarrow ssonoD \Rightarrow ssonorE \Rightarrow ssonoraF \Rightarrow ssonora$

(acepta “ssonora”)

$S \Rightarrow sA \Rightarrow soB \Rightarrow sonC$ (C no deriva ε , rechaza “son”)

Ejercicio 116. Desambigüe la GLC con producciones:

$$\begin{array}{lcl} S & \rightarrow & [S] \\ & | & (S) \\ & | & \{S\} \\ & | & SS \\ & | & \varepsilon \end{array}$$

Gramática desambiguada

Para eliminar la ambigüedad, separamos la gramática en dos niveles: uno que genera un único bloque balanceado (T) y otro que genera una secuencia de dichos bloques (S).

Definimos $G = (V, \Sigma, P, S)$ con $V = \{S, T\}$ y $\Sigma = \{ (,), [,], \{, \} \}$.

Producciones P :

$$\begin{array}{lcl} S & \rightarrow & TS \\ & | & \varepsilon \\ T & \rightarrow & (S) \\ & | & [S] \\ & | & \{S\} \end{array}$$

Ya no es posible elegir arbitrariamente dónde "romper" la cadena SS .

La cadena vacía ε solo es generada por S , evitando que T pueda ser invisible y crear derivaciones redundantes.

Verificación:

Para la cadena $()()$, la única derivación por la izquierda posible es:

$$S \Rightarrow TS \Rightarrow (S)S \Rightarrow (\varepsilon)S \Rightarrow ()TS \Rightarrow ()(S)S \Rightarrow ()(\varepsilon)S \Rightarrow ()() \varepsilon \Rightarrow ()()$$

No existe otra secuencia de reglas que produzca la misma cadena, por lo tanto, la gramática no es ambigua.

Ejercicio 117. La GLC del ejemplo 54 es ambigua. De dos ejemplos de árboles de derivación para una misma cadena.

Ejemplo 54:

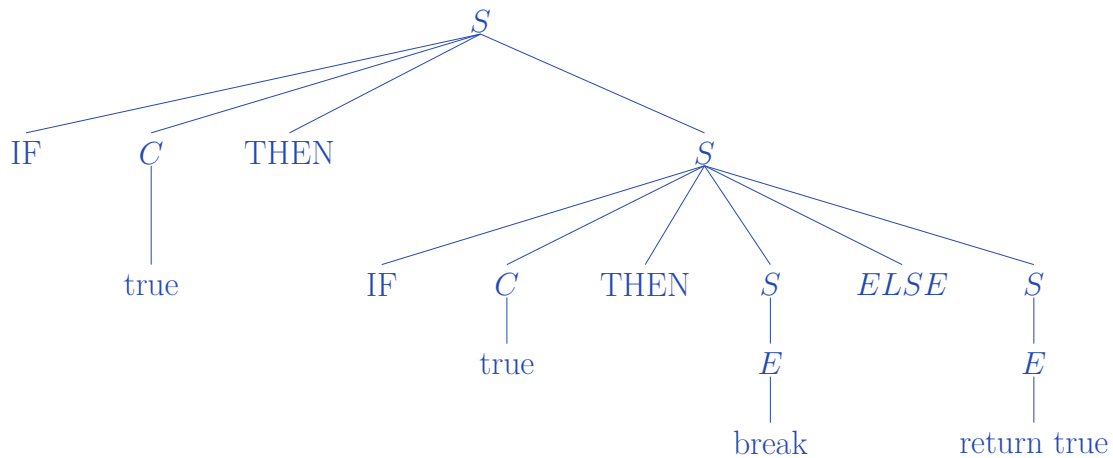
```

S → IF C THEN S
    | IF C THEN S ELSE S
    | E
E → print ".°k"
    | break
    | return true
    | return false
    | E, E
C → false
    | true

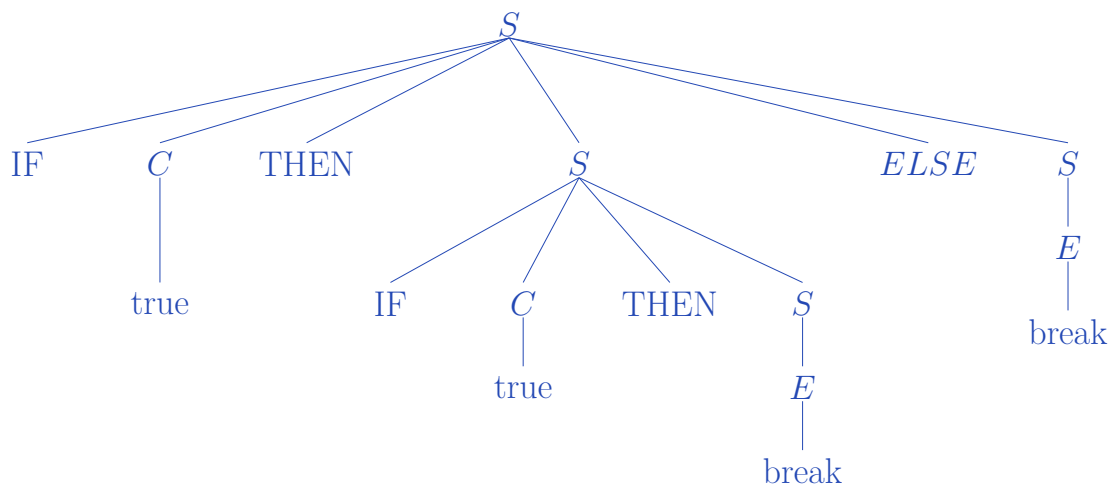
```

Cadena: IF true THEN IF true THEN break ELSE return true

Ejemplo 1



Ejemplo 2



Decide si la siguiente gramática alternativa es ambigua o no:

```
 $S \rightarrow$  IF  $C$  THEN  $S$  ENDIF  
      | IF  $C$  THEN  $S$  ELSE  $S$  ENDIF  
      |  $E$   
 $E \rightarrow$  print .°k"  
      | break  
      | return true  
      | return false  
      |  $E$  print .°k"  
      |  $E$  break  
      |  $E$  return true  
      |  $E$  return false  
 $C \rightarrow$  false  
      | true
```

La gramática alternativa **no es ambigua**.

La ambigüedad original del Ejemplo 54 se debía al problema de la cadena IF true THEN IF true THEN break ELSE return true admitía dos árboles de derivación distintos porque no había forma de determinar a cuál IF pertenecía el ELSE.

En la gramática alternativa, la introducción del token ENDIF como marcador de cierre obligatorio resuelve el problema. Ahora las dos lecturas antes ambiguas producen cadenas **distintas**:

- IF true THEN IF true THEN break ELSE return true ENDIF ENDIF
(el ELSE pertenece al IF interior, que cierra con su propio ENDIF)
- IF true THEN IF true THEN break ENDIF ELSE return true ENDIF
(el ELSE pertenece al IF exterior, pues el IF interior ya cerró con ENDIF)

Al ser cadenas distintas, ninguna admite dos árboles de derivación. En general, dado cualquier IF, el ENDIF correspondiente lo cierra de forma unívoca: si aparece ELSE antes del ENDIF, se aplica la producción $S \rightarrow$ IF C THEN S ELSE S ENDIF; si no, se aplica $S \rightarrow$ IF C THEN S ENDIF. En ambos casos la elección es determinista.

Las producciones de E tampoco introducen ambigüedad: son estrictamente recursivas por la izquierda ($E \rightarrow E$ break, $E \rightarrow E$ return true, etc.), por lo que cada secuencia de terminales derivable desde E tiene una única derivación por izquierda.